TECHNICAL DESIGN DOCUMENT

Shortlist

AI-assisted resume screening and client shortlist portal

A CREDIBLE CORE SLICE, DOCUMENTED HONESTLY

PREPARED FOR

Company A Staffing, Noida

AUTHOR

Ankit Sinha

DATE

November 2026

VERSION

2.0

PUBLIC REPOSITORY

https://github.com/[owner]/shortlist-screening

This document describes the implementation in the repository as built. It does not claim a live deployment, customer adoption or performance results that have not been measured.

Contents

01 Executive summary

02 Brief interpretation and requirement coverage

03 Labelled assumptions

04 Architecture

05 Data model

06 Authentication, roles and tenancy

07 Core workflow

08 Scoring, evidence and uncertainty

09 Client masking

10 DPDP Act 2023 controls

11 Frontend design

12 API surface

13 Security review

14 Testing and verification

15 Trade-offs

16 Deployment summary

17 Demo accounts

18 Limitations and next steps

19 Honest summary

1. Executive summary

Shortlist is a working full-stack prototype for the staffing-firm workflow described in the brief:

1. A recruiter turns a job description into an editable scorecard.
2. CVs are uploaded in bulk, parsed, deduplicated and scored against that scorecard.
3. Every score is backed by a verbatim line from the CV, with explicit uncertainty where the CV is incomplete.
4. A recruiter must open a CV and explicitly verify it before it can be submitted.
5. The client sees only submitted, recruiter-verified profiles, with contact details masked and no raw CV download.
6. The client can shortlist, pass with a reason, or propose interview slots.
7. Access, verification and client decisions are written to an audit log. Candidate erasure anonymises the profile and CV text while retaining the audit trail.

The system is intentionally a **credible core slice**, not a pretend enterprise platform. It uses SQLite, session-backed authentication, deterministic evidence matching and a small number of clearly scoped screens. There are no invented ROI figures, no fake customer logos, no unverified performance claims and no auto-rejection.

1.1 What this document is

This TDD explains the implementation that exists in the repository. Where the brief asks for an outcome the prototype cannot prove (for example, reduced time-to-shortlist or client acceptance rate), this document describes how that outcome would be measured in a pilot rather than claiming it has already happened.

1.2 What was deliberately not built

- Job-board scraping, email ingestion and calendar integrations.
- OCR for scanned PDFs. Scanned files without a text layer are reported as errors.
- A trained ranking model or vector search. Scoring is deterministic and explainable.
- SSO/SAML, background workers, message queues and horizontal scaling.
- Multi-tenant administration beyond the client-company scoping described in section 6.
- A production-grade secrets manager, database encryption at rest or a full DPDP legal compliance programme.

---

2. Brief interpretation and requirement coverage

Brief requirement	Implementation	Evidence
JD scorecard	Pasted JD is converted into an editable scorecard: experience band, notice limit, CTC cap, location, must-have skills and weights. Optional LLM extraction with a deterministic fallback.	`backend/src/services/ai_scorer.js`, `ScorecardPage.tsx`
Bulk CV upload and parsing	Multi-file PDF/DOCX/TXT upload, text extraction, profile parsing, duplicate detection and scoring.	`POST /api/candidates/upload`, `cv_parser.js`, `duplicate_detector.js`
Ranked list with evidence	Every candidate has an overall score and criterion scores. Matched criteria include a verbatim quote and line number; missing evidence is called out, and each candidate carries a confidence level.	`GET /api/candidates`, `CandidateDrawer.tsx`
Client shortlist portal	Client-only portal, scoped to the client company, showing submitted candidates with masked contacts and no raw CV.	`client.js`, `ClientShortlistPage.tsx`

Brief requirement	Implementation	Evidence
Pipeline	Internal stages plus client-driven stages, with the verification gate before submission.	`pipeline.js`, `PipelinePage.tsx`
Sign-in and roles	Real login with scrypt password hashes and HttpOnly session cookies. Three roles: recruiter, client hiring manager, delivery head.	`auth_service.js`, `auth.js`, `middleware/auth.js`
Database	SQLite with WAL mode, schema creation on boot and demo seeding when empty.	`database.js`, `seed.js`
Frontend and backend	React 19 + Vite 8 frontend; Express 5 API.	`frontend/`, `backend/`
AI with uncertainty handling	Deterministic evidence matcher flags missing notice, missing CTC, below-band experience and missing skills. Confidence is high/medium/low.	`ai_scorer.js`
No auto-rejection	Nothing is removed or rejected by the scorer. Low scores remain searchable; rejection is a human or client action.	scorer output, pipeline routes
Recruiter must open CV first	Verification requires an existing `view_unmasked_cv` audit entry for that user, candidate and requisition.	`POST /api/candidates/:id/verify`
Masked client contacts	Client response masks name, email and phone; it does not include `raw_cv_text`.	`pii_masker.js`, `client.js`
DPDP consent/deletion/audit	Consent fields per candidate, append-style audit log, right-to-erasure workflow with receipt.	`dpdp_service.js`, `dpdp.js`, `CompliancePage.tsx`
Mobile-polished UI	Responsive layout with mobile drawer, stacked cards, mobile filters and client portal layouts.	`index.css`, screenshots captured during verification
Loading/empty/error states	Skeleton loaders, empty states, inline errors and toast notifications.	`common.tsx`, page components
Tests	25 automated backend tests covering services, auth, tenancy, verification, upload and erasure.	`backend/tests/all_tests.js`
No secrets	`.env.example` contains template values only; real keys and `.env` files are ignored.	`.gitignore`, `.env.example`

3. Labelled assumptions

- **A1 — Demo data is fictional.** All names, CVs, companies and contact details in the seed are synthetic. The client names are used only to exercise tenant isolation.
- **A2 — Consent is recorded, not legally engineered.** Uploads store a consent status, date and purpose. The prototype does not implement a candidate-facing consent capture journey or consent withdrawal UI.
- **A3 — Scorecard defaults are a starting point.** Default weights are skills 40%, experience 30%, notice 15%, CTC 15%. Recruiters can change them. The weights must add up to 100%.
- **A4 — Notice and CTC are often missing.** Missing values are scored neutrally and flagged, never guessed and never treated as a rejection reason.
- **A5 — The client contact mask is partial by design.** The client sees a shortened name and masked email/phone. The recruiter retains the full profile internally.
- **A6 — SQLite is deliberate.** It keeps the review environment reproducible. A production deployment should move to PostgreSQL or another managed database before concurrent multi-instance use.

- **A7 — AI is optional and bounded.** The only external LLM call is JD-to-scorecard extraction. CV scoring is local and deterministic. If no API key is configured, the local JD extractor is used.

---

4. Architecture

4.1 Runtime topology

```
Browser
  React 19 + Vite 8 SPA
  HttpOnly session cookie
        |
        | HTTPS/JSON in production
        v
Express 5 API
  auth middleware -> role guard -> tenant guard
  routes: auth, requisitions, candidates, pipeline, client, dpdp, analytics
        |
        v
SQLite (better-sqlite3, WAL)
  users, sessions, requisitions, candidates,
  cv_evaluations, access_audit_logs, calibration_audits
```

In development, Vite runs on port 5173 and proxies `/api` to the API on port 4000. In production, the API serves the built frontend from `frontend/dist` on port 4000.

4.2 Technology choices

Layer	Choice	Why
Frontend	React 19, TypeScript, Vite 8	Fast feedback, typed domain objects, simple SPA routing
Styling	Custom CSS design system	Small dependency surface, full control of responsive behaviour
API	Node.js 22, Express 5	Familiar HTTP model, straightforward middleware and file upload handling
Database	SQLite via `better-sqlite3`, WAL	Zero-setup reviewer experience and transactional local writes
Auth	`scrypt` password hashes, random session tokens	No plaintext passwords, no spoofable user-id headers
Parsing	`pdf-parse`, `mammoth`	Server-side text extraction without writing uploaded files to disk
Tests	Node test runner + Supertest	No test framework dependency beyond the API client
Packaging	Dockerfile + compose	One-command review environment with a persistent SQLite volume

4.3 Why SQLite only

The original project included an unused PostgreSQL branch and a committed database file. Both were removed. Keeping a second, untested database engine would have made the deployment story less credible. SQLite is used consistently, the schema is created on boot, and the test suite uses a disposable database file.

The trade-off is real: SQLite is not the right choice for many concurrent API instances or for a managed production database. `deployment.md` states this explicitly.

---

5. Data model

The schema is defined in `backend/src/db/database.js`. The main tables are:

`users`

- `id`, `email` (unique), `name`, `role`, `company_id`, `company_name`
- `password_hash` in the form `scrypt$<salt>$<derived-key>`
- Roles: `recruiter`, `client_hiring_manager`, `delivery_head`

`sessions`

- `id` = SHA-256 hash of the random bearer token
- `user_id`, `expires_at`, `created_at`
- Seven-day expiry, deleted on logout and purged opportunistically on login

`requisitions`

- `id`, `title`, `department`, `client_company`, `client_contact_name`, `client_user_id`
- `status`, `openings_count`, experience band, CTC cap, notice limit, location
- `raw_jd`, `scorecard_json`, `created_by`, timestamps
- The scorecard is the single source of truth for scoring weights and criteria.

`candidates`

- `id` (`CND-xxxxx`), name, email, phone, current role, total experience
- `key_skills_json`, current/expected CTC, notice days, location
- `raw_cv_text`, `original_filename`, `file_hash`
- Consent status/date/purpose, `is_deleted`, `deleted_at`

`cv_evaluations`

- One row per candidate per requisition
- Criterion scores, overall score, `breakdown_json`, `ai_confidence`, uncertainty notes
- `scoring_engine`, recruiter verification fields and notes
- `pipeline_stage`, rejection fields, client feedback, interview slots

`access_audit_logs`

- Append-only rows for CV views, verification, stage changes, uploads, client decisions, scorecard changes and erasure
- `user_id`, `user_name`, `user_role`, candidate/requisition references, action, IP, JSON details, timestamp

`calibration_audits`

- Delivery-head sample comparing AI rank with recruiter rank
- Week label, ranks, discrepancy, agreement status, override reason, auditor

Indexes and integrity

Foreign keys are enabled. The application performs explicit lookups by `requisition_id`, `candidate_id` and `user_id`. The test suite exercises cascade/cleanup behaviour through the erasure path.

---

6. Authentication, roles and tenancy

6.1 Sign-in

- `POST /api/auth/login` accepts email and password.
- Passwords are hashed with `scrypt` and compared with `timingSafeEqual`.
- A successful login creates a random 32-byte token, stores only its SHA-256 hash in `sessions`, and sets `shortlist_session` as an HttpOnly, SameSite=Lax cookie. `Secure` is enabled when `NODE_ENV=production`.
- `GET /api/auth/me` restores the session on page load.
- `POST /api/auth/logout` deletes the session and clears the cookie.
- Failed logins are limited to 12 attempts per IP per 10-minute window. This is an in-memory, single-process limiter; a production system needs a shared store.

The old `x-user-id` header path was removed because it allowed any caller to impersonate a role.

6.2 Role matrix

Capability	Recruiter	Client hiring manager	Delivery head
View internal candidates, CVs and pipeline	Yes	No	Yes
Create/edit requisitions and scorecards	Yes	No	Yes
Upload CVs	Yes	No	Yes
Verify CVs	Yes	No	Yes
Submit to client	Yes	No	Yes
View client portal	No	Own company only	View via internal routes only
Shortlist/pass/propose slots	No	Own company only	No
View audit/consent records	Yes	No	Yes
Erase candidate	Yes	No	Yes
Record calibration samples	No	No	Yes

6.3 Tenant isolation

A client user has a `company_name`. Every client requisition carries a `client_company`. The guard checks:

```
requisition.client_company === user.company_name
```

A client cannot list or open another client's requisition, even by guessing the requisition ID. Internal routes are restricted to `recruiter` and `delivery_head`.

This is a simple, explicit tenancy model for the prototype. It is not a substitute for row-level security in a multi-tenant production database.

---

7. Core workflow

7.1 Scorecard creation

1. A recruiter pastes a JD and an optional title.
2. `POST /api/requisitions/intake-jd` calls `extractScorecardWithAI`.
3. If an LLM key is configured and the response is valid, the LLM result is normalised. Otherwise the deterministic extractor parses experience, notice, CTC, location and a small skills taxonomy.
4. The recruiter edits every field in the UI, including skills and weights.
5. `POST /api/requisitions` validates the scorecard and creates the requisition.
6. `PUT /api/requisitions/:id/scorecard` saves changes and re-scores every existing evaluation inside a transaction.

Validation includes: title present, experience between 0 and 40 years, max CTC between 1 and 200 LPA, notice between 0 and 180 days, 1–20 skills, each skill name non-empty, weights between 0 and 1 and summing to 100% within a small tolerance.

7.2 Bulk upload

1. The recruiter selects a requisition and drops PDF/DOCX/TXT files.
2. Multer stores files in memory; nothing is written to a temporary upload directory.
3. Limits: 10 MB per file, 500 files per batch, 200 MB total, extension allow-list.
4. Each file is extracted to text:
   - PDF: `pdf-parse`
   - DOCX: `mammoth`
   - TXT: UTF-8
5. Files with less than 30 characters of text are reported as unreadable (typically scanned PDFs).
6. Duplicate detection checks, in order: SHA-256 of the extracted text, normalised email, last ten digits of phone.
7. A duplicate profile already in the requisition is reported. If the profile exists but is not in this requisition, it is scored for this requisition without creating a duplicate candidate.
8. New profiles are inserted, scored and placed in `ai_screened`.
9. The response is a batch report: added, duplicates, errors, per-file warnings.

7.3 Recruiter review and verification

1. The drawer calls `GET /api/candidates/:id?requisitionId=...`.
2. That request writes `view_unmasked_cv` to the audit log.
3. The drawer shows the score, confidence, per-criterion evidence and the full CV text.
4. `POST /api/candidates/:id/verify` checks for a prior `view_unmasked_cv` entry for the same user, candidate and requisition. Without it, the request returns 400.
5. Verification records the reviewer name and notes, moves `ai_screened` to `recruiter_verified`, and writes an audit entry.

This prevents a bulk “verify all” shortcut from being implemented without an actual review.

7.4 Submission and pipeline

- `POST /api/pipeline/move` allows internal stages only.
- Submission to `submitted_to_client` requires `recruiter_verified = 1`.
- Client-only stages (`client_shortlisted`, `interview_scheduled`) cannot be set from the internal workspace.
- `POST /api/pipeline/batch-submit` exists for bulk operations but still blocks unverified candidates and returns a per-candidate reason.
- The client portal is the only place where client decisions are recorded.

7.5 Client portal

- `GET /api/client/requisitions` returns only the client's requisitions.
- `GET /api/client/shortlist/:requisitionId` returns only candidates in submitted-or-later stages.
- The response masks name, email and phone and omits `raw_cv_text`.
- `POST /api/client/action` supports:
  - `shortlist`
  - `reject` with a reason and optional notes
  - `propose_slots` with one to three validated date/time/round entries
- Every action is audit-logged and limited to the client's own requisition.

---

8. Scoring, evidence and uncertainty

8.1 Scope of “AI”

The scoring engine is a deterministic evidence matcher, not a black-box model. This was a deliberate choice:

- Every score can be explained by a line in the CV.
- Results are reproducible without an API key, network access or model version drift.
- The prototype does not claim semantic understanding it does not have.
- An optional LLM is used only to turn a JD into an editable scorecard. Reviewers can correct the result.

8.2 Formula

For each candidate and scorecard:

```
overall = round(
  skills_score × skills_weight +
  experience_score × experience_weight +
  notice_score × notice_weight +
  ctc_score × ctc_weight
)
```

Weights are editable and must total 100%.

8.3 Criterion rules

Must-have skills

- Each skill is searched as a whole word in the CV text.

- When found, the matching line and line number are stored as evidence and the criterion scores 100.
- If the skill appears only in the candidate's structured skills list, it scores 85 with medium confidence and a note that no project-level evidence was found.
- If absent, the criterion scores 0 and the skill is listed as a gap.
- Minimum years attached to a skill are displayed, but the prototype does **not** independently verify years from free text. The UI says so.

Experience

- Inside the requested band: 100.
- Above the band: `max(70, 100 − 5 × years_over)` with an overqualification note.
- Below the band: `max(20, 100 − 25 × years_under)` with medium confidence and a gap note.
- An evidence line mentioning experience is included when available.

Notice period

- Missing: neutral 50, low confidence, explicit “recruiter must confirm” note.
- Within limit: 100.
- Over limit by up to 30 days: 60.
- More than 30 days over: 35.
- The candidate is never rejected because of notice.

Compensation

- Missing: neutral 50, low confidence, explicit confirmation note.
- Within budget: 100.
- Over budget: `max(30, round(100 − 2 × percentage_over))`.

Overall confidence

- Any low-confidence criterion, or two or more uncertainty flags: `low`.
- Exactly one uncertainty flag: `medium`.
- Otherwise: `high`.

8.4 Anti-auto-rejection

The scorer returns a score and explanations. It does not delete, hide, reject or rank-out any candidate. Low scorers remain in the ranked list and are searchable and filterable. Rejection is an explicit action by a recruiter or client and is recorded in the audit log.

8.5 Known limitations

- Whole-word matching misses synonyms and implied evidence (“built payment rails” does not match “Kafka”).
- The skills taxonomy is finite.
- Years of experience per skill are not verified from the CV.
- Parsing is heuristic and can misread unusual CV layouts.
- The LLM path is not used for scoring, so there is no semantic similarity layer.

These limitations are surfaced in the UI and in the README rather than hidden behind an “AI-powered” claim.

---

9. Client masking

`pii_masker.js` transforms a candidate for the client portal:

- Name: first name plus initial, e.g. `Ritika B.`
- Email: first character plus `***` plus domain, e.g. `r***2@gmail.com`
- Phone: country code and last two digits, e.g. `+91 98*** ***90`

The client response contains evidence quotes from the evaluation but not the raw CV. The client cannot download the CV, and contact details are not revealed by the API response.

Masking is a product control, not encryption. A production system should also consider field-level encryption and a formal disclosure policy.

---

10. DPDP Act 2023 controls

The prototype implements a narrow, testable subset of DPDP obligations:

10.1 Consent records

Each candidate stores:

- `consent_status` (`consented` / `deleted`)
- `consent_date`
- `consent_purpose` (the client, role and lawful purpose)

The demo upload path marks uploaded CVs as consented for the selected requisition. This is a demo convenience, not a substitute for a candidate-facing notice and consent journey.

10.2 Access audit

`logAccess` writes an append-only audit row for:

- CV opens (`view_unmasked_cv`)
- Verification (`recruiter_verified`)
- Pipeline movement (`stage_change`)
- Bulk uploads (`bulk_cv_upload`)
- Client portal views and decisions
- Scorecard changes and requisition creation
- Candidate erasure (`candidate_deletion`)

The compliance screen supports filtering by action and searching the visible log. Audit rows are retained when a candidate is erased.

10.3 Right to erasure

`POST /api/dpdp/erasure-request` :

1. Replaces name, email and phone with anonymised values.
2. Clears role, experience, skills, CTC, notice, location, original filename and file hash.

3. Replaces `raw_cv_text` with a timestamped erasure tombstone.
4. Clears evaluation evidence, notes, rejection text and interview slots.
5. Marks the candidate deleted and consent withdrawn.
6. Writes a `candidate_deletion` audit row.
7. Returns a purge receipt such as `DPDP-PURGE-XXXXXXXXXXXX`.

The audit trail survives, but direct identifiers and CV content do not. This is the tested behaviour in `API DPDP - erasure anonymises the candidate and retains the audit trail`.

10.4 What is not implemented

- Candidate-facing consent capture or withdrawal.
- Data-retention schedules and automated expiry.
- Encryption at rest or field-level encryption.
- Data-processor agreements, breach notification workflow or a DPO portal.
- Cross-border transfer controls.

Those are production compliance workstreams, not claims made by this prototype.

---

11. Frontend design

11.1 Information architecture

The previous seven-tab layout was replaced with a role-based workspace:

Recruiter

- Candidates
- Add CVs
- Pipeline
- Scorecard
- Compliance

Client hiring manager

- Shortlist only

Delivery head

- Candidates, Add CVs, Pipeline, Scorecard, Insights, Compliance

The requisition picker is always visible so the user knows which scorecard and pipeline they are working in.

11.2 Interaction details

- Session bootstrap via `/api/auth/me`; no flash of a fake workspace.
- Sign-in screen lists seeded demo accounts without passwords.
- Candidate list: search, stage filter, score filter, blind-review toggle.
- Candidate drawer: evidence and full-CV tabs, confidence callout, criterion cards, quote and line number, verification notes, verify and submit actions.

- Upload: drag-and-drop, file queue, sample CV generator, batch report with added/duplicate/error counts.
- Pipeline: five columns (To review, Verified, With client, Interviewing, Closed) with verification-aware actions.
- Scorecard: editor with skills, weights, reset/balance helpers and live weight total.
- Compliance: audit trail, consent registry, erasure action and receipt.
- Client portal: masked cards, evidence disclosure, shortlist/pass/slot actions, rejected-state feedback.

11.3 Responsive behaviour

- Sidebar becomes an off-canvas drawer below 1020 px.
- Mobile top bar uses a three-column grid for menu, title and user chip, with the requisition picker on its own row.
- Candidate stats become a two-column grid; filters stack into a search row, two selects and a full-width blind-review control.
- Client portal cards stack their metadata and actions.
- Tables scroll horizontally rather than crushing columns.

11.4 States

- Loading: skeleton rows and drawer skeletons.
- Empty: “no candidates match these filters”, “no requisitions yet”, “no client shortlist”.
- Error: inline error messages and toasts; API errors are user-readable.
- Feedback: success/error/info toasts; verification and submission buttons show progress text.

---

12. API surface

All routes below are prefixed with `/api`. Authentication is a session cookie or a bearer token.

Method	Route	Access	Purpose
GET	`/health`	Public	Health check
GET	`/auth/demo-accounts`	Public	Seeded demo users, no passwords
POST	`/auth/login`	Public	Sign in
GET	`/auth/me`	Signed in	Session bootstrap
POST	`/auth/logout`	Signed in	Destroy session
GET	`/requisitions`	Internal	List requisitions
GET	`/requisitions/:id`	Internal	Requisition and scorecard
POST	`/requisitions/intake-jd`	Internal	JD to scorecard
POST	`/requisitions`	Internal	Create requisition
PUT	`/requisitions/:id/scorecard`	Internal	Save and re-score
GET	`/candidates`	Internal	Ranked/filterable candidate list
GET	`/candidates/:id`	Internal	Full CV and evidence; writes audit log
POST	`/candidates/:id/verify`	Internal	Verification after a recorded view

Method	Route	Access	Purpose
POST	`/candidates/upload`	Internal	Bulk upload and scoring
GET	`/pipeline/:requisitionId`	Internal	Pipeline board
POST	`/pipeline/move`	Internal	Move internal stage
POST	`/pipeline/batch-submit`	Internal	Submit verified candidates
GET	`/client/requisitions`	Client	Own requisitions
GET	`/client/shortlist/:requisitionId`	Client	Masked submitted profiles
POST	`/client/action`	Client	Shortlist, reject, propose slots
GET	`/dpdp/audit-logs`	Internal	Audit trail
GET	`/dpdp/consents`	Internal	Consent registry
POST	`/dpdp/erasure-request`	Internal	Erase candidate
GET	`/analytics/kpi-metrics`	Internal	Real counts, no modelled ROI
GET	`/analytics/calibration-audits`	Internal	Delivery-head samples
POST	`/analytics/calibration-audits`	Delivery head	Record sample

Inputs are length-limited and validated. SQL uses prepared statements. Error responses avoid leaking stack traces.

13. Security review

Risk	Control	Residual risk
Spoofed identity	Session tokens, HttpOnly cookie, scrypt hashes	No MFA, no SSO
Brute force	Per-IP login limit, constant-time failure path	In-memory limiter resets on restart and is per-process
Cross-client data access	Company-name tenant guard on every client route	Simple string equality, not a hardened multi-tenant model
Cross-origin credential use	Production allows only `CORS_ORIGIN` entries; same-origin requests work without CORS	A split-origin deployment must configure the allow-list correctly
Recruiter rubber-stamping	Verification requires a recorded CV view	A recruiter can still open and immediately verify; UI notes are encouraged but not a quality SLA
Upload abuse	Extension allow-list, MIME/size limits, memory storage	No malware scanning
Prompt injection via JD	LLM only extracts a scorecard; result is editable and normalised	A malicious JD could still influence extracted fields
XSS	React escaping; no `dangerouslySetInnerHTML` in the app	No formal CSP
CSV/PDF injection	Client portal exposes no raw CV download	Not applicable to this build
Secrets	`.env` ignored; example contains names only	Production needs a secret manager

14. Testing and verification

14.1 Automated tests

Command:

```
cd backend
npm test
```

Result on the final build:

```
1..25
# tests 25
# pass 25
# fail 0
```

Coverage areas:

- CV parser: contacts, experience, notice, CTC, missing values, immediate joiners, no fake contacts.
- Scorer: verbatim evidence, missing notice/CTC uncertainty, JD extraction, blind screening.
- PII masking for the client portal.
- Duplicate detection by hash and contact details.
- Public health check and anonymous rejection on protected routes.
- Demo accounts never expose password hashes.
- HttpOnly session cookie on login and clearing on logout.
- Login rate limiting and success reset.
- Client role cannot access internal workspaces.
- Client tenancy isolation.
- Client portal masks contacts and never returns raw CV text.
- Client-only decisions and input validation.
- CV must be opened before verification.
- Unverified candidates cannot be submitted.
- Client-only stages cannot be moved internally.
- Requisition JD intake and scorecard validation.
- Bulk upload parsing, scoring, deduplication and unsafe file rejection.
- DPDP erasure anonymises the candidate and retains the audit trail.

The suite uses `data/test-screening.db` and deletes it before and after the run. It never touches the demo database.

14.2 Manual verification performed

- Signed in as each of the four demo users.
- Created the scorecard from a JD and saved/re-scored candidates.
- Uploaded three generated sample CVs and reviewed the batch report.
- Opened an AI-screened candidate, verified the CV and submitted to the client.
- Shortlisted, passed and proposed slots from the client portal.
- Erased a candidate and confirmed the receipt and audit entry.

- Captured desktop and mobile screenshots of all main screens.
- Checked that the production build succeeds and TypeScript reports no errors.

14.3 What is not tested

- Real PDF/DOCX files with complex layouts.
- OCR or image-only PDFs.
- Load, concurrency or multi-instance behaviour.
- Browser accessibility audit with a screen reader.
- Full end-to-end browser automation in CI (manual CDP checks were used during development).

---

15. Trade-offs

Decision	Benefit	Cost
Deterministic scorer instead of an LLM for CVs	Reproducible, auditable, offline	Misses synonyms and implied evidence
SQLite instead of PostgreSQL	Zero-setup review, simple deployment	Not suitable for concurrent production scale
Session cookies instead of header identity	Real authentication, no spoofing	Requires cookie/CORS configuration in deployment
Verification gate based on an audit view	Enforces a human step	Adds a click and depends on audit availability
Masking in the client API	Reduces disintermediation risk	Partial mask is not a legal guarantee
Delete the unused PostgreSQL branch	Smaller surface, honest docs	One engine to operate, no horizontal scale story
No fake ROI dashboard	Credible	Less impressive-looking than invented metrics

---

16. Deployment summary

See `deployment.md` for the complete runbook. In short:

```
npm install
npm --prefix backend install
npm --prefix frontend install
npm test
npm run dev
```

Production:

```
npm run build
npm start
# http://localhost:4000
```

Docker:

```
docker compose up --build
# http://localhost:4000
```

The SQLite file lives at `backend/data/screening.db` by default. Mount that directory as a volume in Docker. The database is created and seeded automatically when missing.

---

17. Demo accounts

All demo accounts use the password `demo1234`.

Role	Name	Email	Scope
Recruiter	Priya Sharma	`priya@companya.in`	Company A internal workspace
Client hiring manager	Vikram Malhotra	`vikram@hdfctech.com`	HDFC Tech Platform only
Client hiring manager	Ananya Sen	`ananya@standardchartered.com`	Standard Chartered only
Delivery head	Rajesh Verma	`rajesh@companya.in`	Internal + calibration + reset demo data

The delivery-head account can reset the demo dataset from the Insights screen/API. This is a demo convenience, not a production administration feature.

---

18. Limitations and next steps

Limitations

- Heuristic parsing and matching, not a trained ranking model.
- No OCR.
- No email/calendar integration.
- No background job queue; uploads are processed in the request.
- Audit logs are append-only by application convention, not by database permissions.
- Consent capture is a demo record, not a legal workflow.
- SQLite and in-memory rate limiting are single-instance choices.
- No automated browser test suite.

Next steps

1. Add a candidate-facing consent and withdrawal journey.
2. Add OCR for scanned CVs and a parser confidence report.
3. Add semantic matching as a **secondary** signal, with the verbatim evidence view unchanged.
4. Move to PostgreSQL and add row-level security for client tenancy.
5. Add Playwright end-to-end tests for sign-in, upload, verification and client decisions.
6. Add retention policies, encryption at rest and a DPO reporting view.

7. Measure pilot outcomes — time per shortlist, verification rate, client acceptance, override rate — before making any ROI claim.

---

19. Honest summary

This build is strongest where trust matters: a real sign-in, a clear role boundary, a verification gate that cannot be bypassed by a bulk request, evidence quoted from the CV, explicit uncertainty, masked client contacts and an auditable erasure path.

It is not a production ATS. It is a coherent, tested slice that demonstrates the workflow and the design judgement behind it, with the limitations written down instead of hidden.